In [62]:
```
!pip install graphviz
```

```
Requirement already satisfied: graphviz in /opt/anaconda3/lib/python3.13/sit
e-packages (0.21)
```

In [31]:
```
import graphviz
from sklearn import tree
import pandas as pd
import numpy as np
```

In [32]:
```
credit = pd.read_csv('credit.csv')
```

In [33]:
```
credit
```

Out[33]:

	checking_balance	months_loan_duration	credit_history	purpose	amo
0	< 0 DM	6	critical	furniture/appliances	1
1	1 - 200 DM	48	good	furniture/appliances	5
2	unknown	12	critical	education	2
3	< 0 DM	42	good	furniture/appliances	7
4	< 0 DM	24	poor	car	4
...	...	...	...	...	
995	unknown	12	good	furniture/appliances	1
996	< 0 DM	30	good	car	3
997	unknown	12	good	furniture/appliances	
998	< 0 DM	45	good	furniture/appliances	1
999	1 - 200 DM	45	critical	car	4

1000 rows × 17 columns

In [34]:
```
credit.dtypes
```

Out[34]:
```
checking_balance        object
months_loan_duration     int64
credit_history          object
purpose                 object
amount                   int64
savings_balance         object
employment_duration     object
percent_of_income        int64
years_at_residence       int64
age                      int64
other_credit            object
housing                 object
existing_loans_count     int64
job                     object
dependents               int64
phone                   object
default                 object
dtype: object
```

In [35]:
```
credit[['CB1_200','CB0','CBmayor200','CBunknown']] = pd.get_dummies(credit["
credit[['CHcritical','CHgood','CHperfect','CHpoor','CHverygood']] = pd.get_d
credit[['SB100_500','SB500_1000','SBmenor100','SBmayor1000','SBunkown']] = p
credit[['ED1_4','ED4_7','EDmenor1','EDmayor7','EDunemployed']] = pd.get_dumm
credit[["Hother","Hown","Hrent"]] = pd.get_dummies(credit["housing"])
credit[["Jmanagement","Jskilled","Junemployed","Junskilled"]] = pd.get_dummi
credit[["Pno","Pyes"]] = pd.get_dummies(credit["phone"])
credit["y"] = np.where(credit["default"]=="no",0,1)
```

In [36]:
```
credit.dtypes
```

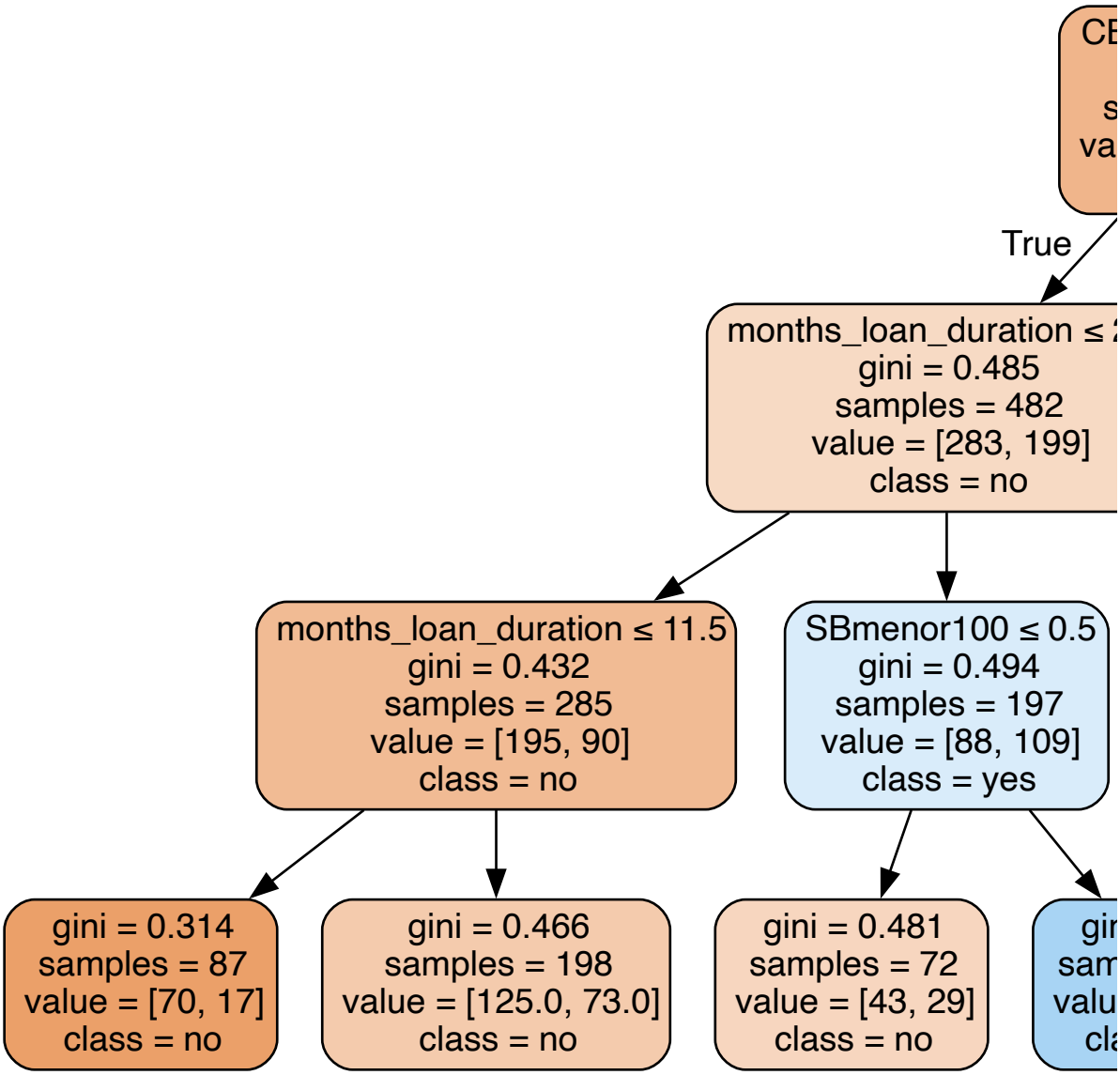
Out[36]:
```
checking_balance        object
months_loan_duration     int64
credit_history          object
purpose                 object
amount                   int64
savings_balance         object
employment_duration     object
percent_of_income        int64
years_at_residence       int64
age                      int64
other_credit            object
housing                 object
existing_loans_count     int64
job                     object
dependents               int64
phone                   object
default                 object
CB1_200                   bool
CB0                       bool
CBmayor200                bool
CBunknown                 bool
CHcritical                bool
CHgood                    bool
CHperfect                 bool
CHpoor                    bool
CHverygood                bool
SB100_500                 bool
SB500_1000                bool
SBmenor100                bool
SBmayor1000               bool
SBunkown                  bool
ED1_4                     bool
ED4_7                     bool
EDmenor1                  bool
EDmayor7                  bool
EDunemployed              bool
Hother                    bool
Hown                      bool
Hrent                     bool
Jmanagement               bool
Jskilled                  bool
Junemployed               bool
Junskilled                bool
Pno                       bool
Pyes                      bool
y                        int64
dtype: object
```

In [37]:
```
from sklearn.model_selection import train_test_split
x_credit=credit.drop(['checking_balance', 'credit_history', 'purpose', 'savi
y_credit=credit["y"]
x_train,x_test,y_train, y_test=train_test_split(x_credit, y_credit,train_siz
```

In [63]:
```
clf = tree.DecisionTreeClassifier(criterion="gini", min_samples_leaf = 50, m
dtree = clf.fit(x_train, y_train)
```

```
dot_data = tree.export_graphviz(dtree,
                                out_file=None,
                                feature_names=x_credit.columns,
                                class_names=["no","yes"],
                                proportion=False,
                                filled=True,
                                rounded=True,
                                special_characters=True)

graph = graphviz.Source(dot_data)
graph
```

Out[63]:



In [43]:

```
from sklearn.metrics import confusion_matrix, classification_report
predicciones = dtree.predict(x_test)
cm=confusion_matrix(y_true=y_test, y_pred=np.round(predicciones,0))
cm
```

Out[43]:
```
array([[116,  19],
       [ 41,  24]])
```

In [45]:
```
print(classification_report(y_test,predicciones))
```

```
              precision    recall  f1-score   support

           0       0.74      0.86      0.79       135
           1       0.56      0.37      0.44        65

    accuracy                           0.70       200
   macro avg       0.65      0.61      0.62       200
weighted avg       0.68      0.70      0.68       200
```

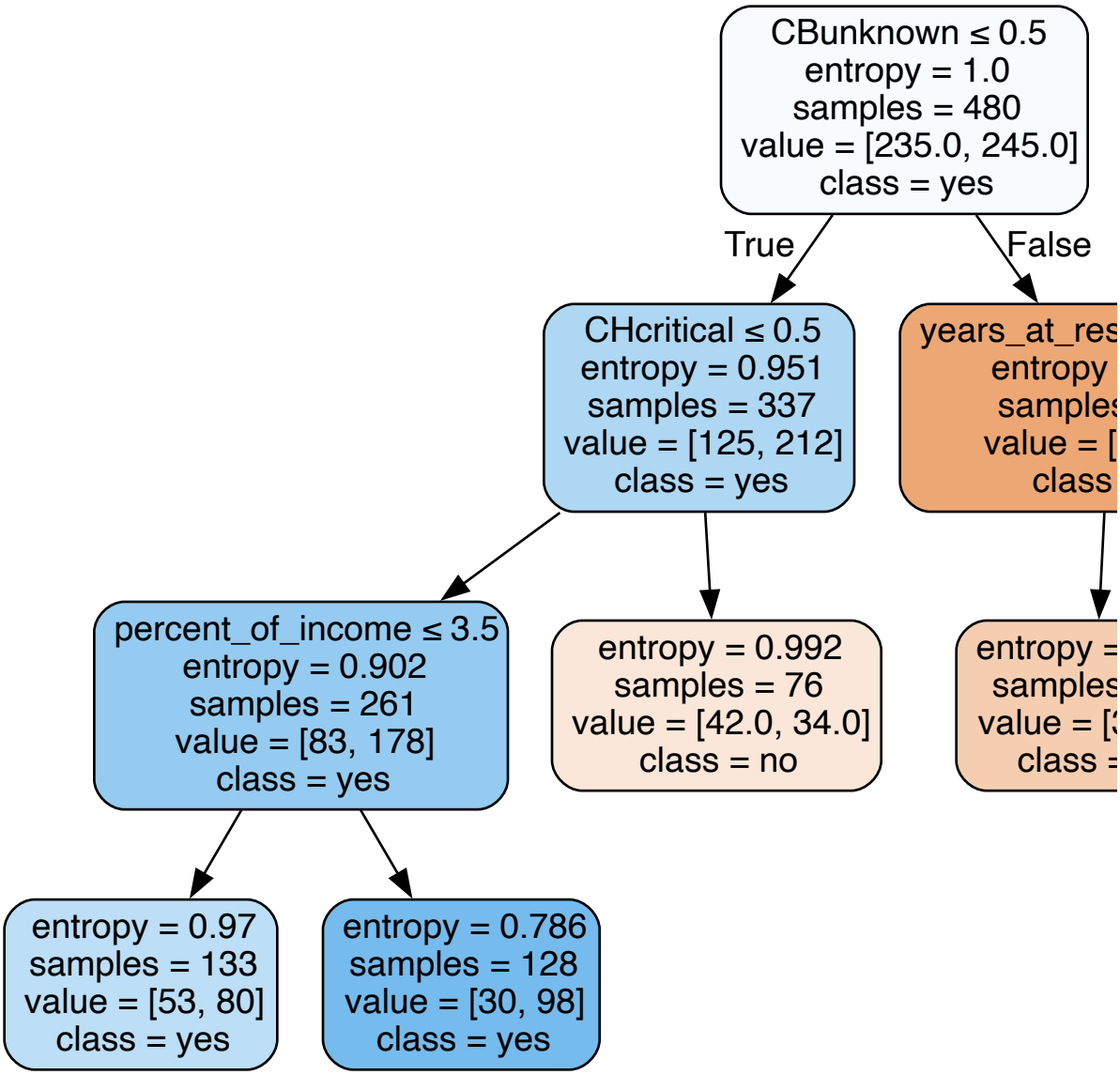
In [47]:
```
credit["default"].value_counts()
```

Out[47]:
```
default
no     700
yes    300
Name: count, dtype: int64
```

In [64]:
```
num_y = credit["default"].value_counts()

min_num = min(num_y)

min_class = num_y[num_y == min(num_y)].index[0]

undersamp = credit[credit["default"] != min_class].sample(min_num, random_st

credit_balanced = pd.concat([undersamp, credit[credit["default"] == min_clas

x_balanced = credit_balanced.drop(
    ['checking_balance','credit_history','purpose','savings_balance',
     'employment_duration','other_credit','housing','job','phone','default',
    axis=1
)

y_balanced = credit_balanced["y"]

x_trainB, x_testB, y_trainB, y_testB = train_test_split(
    x_balanced, y_balanced, train_size=0.8, random_state=14
)

clf = tree.DecisionTreeClassifier(
    criterion="entropy",
    min_samples_leaf=50,
    max_depth=3
)

dtree = clf.fit(x_trainB, y_trainB)

dot_data = tree.export_graphviz(
    dtree,
    out_file=None,
    feature_names=x_credit.columns,
```

```
    class_names=["no", "yes"],
    proportion=False,
    filled=True,
    rounded=True,
    special_characters=True
)

graph = graphviz.Source(dot_data)
graph
```

Out[64]:



In [65]:
```
credit_balanced["default"].value_counts()
```

Out[65]:
```
default
no     300
yes    300
Name: count, dtype: int64
```

In [66]:
```
prediccionesB = dtree.predict(x_testB)
cm = confusion_matrix(y_true=y_testB, y_pred=np.round(prediccionesB,0))
```

```
print(cm)
```

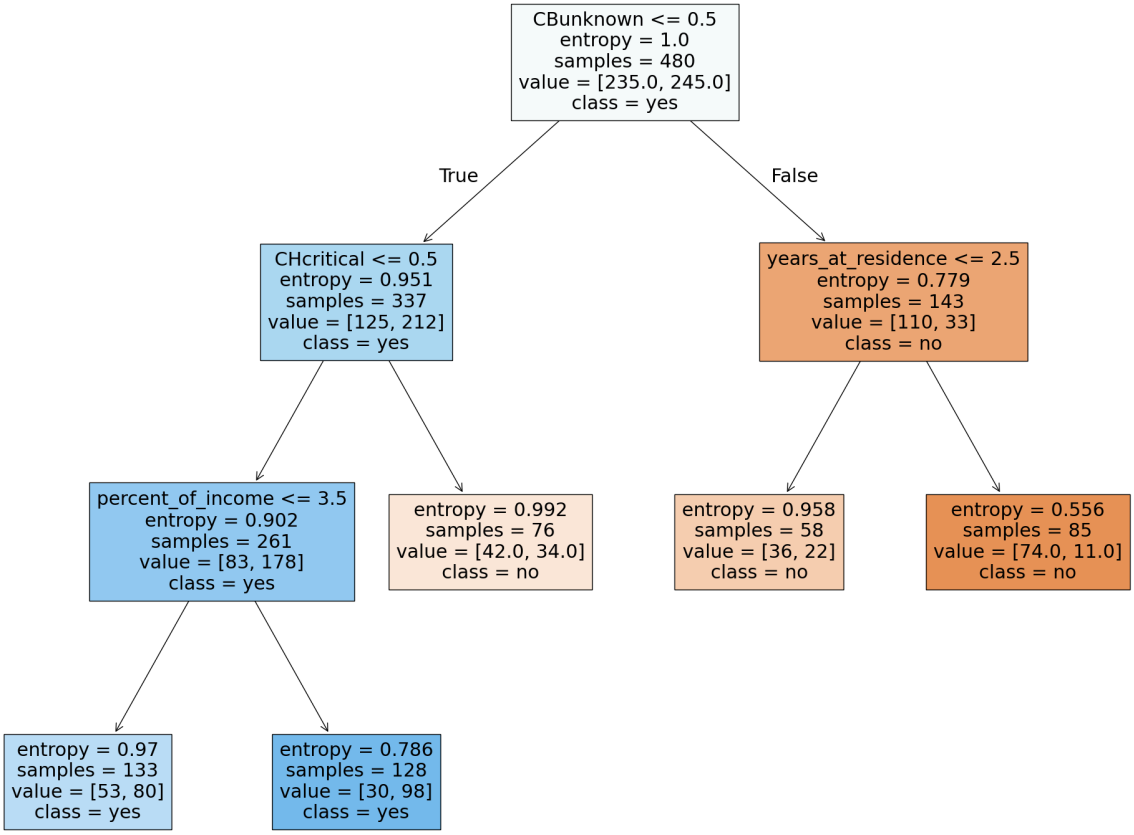
```
[[45 20]
 [19 36]]
```

In [55]:
```
print(classification_report(y_testB,prediccionesB))
```

```
              precision    recall  f1-score   support

           0       0.70      0.69      0.70        65
           1       0.64      0.65      0.65        55

    accuracy                           0.68       120
   macro avg       0.67      0.67      0.67       120
weighted avg       0.68      0.68      0.68       120
```

In [69]:
```
from matplotlib import pyplot as plt
```

In [72]:
```
from sklearn.tree import plot_tree
import matplotlib.pyplot as plt

plt.figure(figsize=(25,20))
plot_tree(
    dtree,
    feature_names=x_balanced.columns,
    class_names=["no","yes"],
    filled=True
)
plt.show()
```


CBunknown <= 0.5
entropy = 1.0
samples = 480
value = [235.0, 245.0]
class = yes
True
False
CHcritical <= 0.5
entropy = 0.951
samples = 337
value = [125, 212]
class = yes
years_at_residence <= 2.5
entropy = 0.779
samples = 143
value = [110, 33]
class = no
percent_of_income <= 3.5
entropy = 0.902
samples = 261
value = [83, 178]
class = yes
entropy = 0.992
samples = 76
value = [42.0, 34.0]
class = no
entropy = 0.958
samples = 58
value = [36, 22]
class = no
entropy = 0.556
samples = 85
value = [74.0, 11.0]
class = no
entropy = 0.97
samples = 133
value = [53, 80]
class = yes
entropy = 0.786
samples = 128
value = [30, 98]
class = yes